Índices, ALTER TABLE e privilégios e Transações

Resumo teórico e expositivo • Continuação do Módulo 15- parte 2

Contexto: BD ClinicaMedica, com: Paciente, Medico, Seguros, Consulta, Consulta_Seguro, Seguro_Medico e Paciente_Seguros.

Os exemplos usam MySQL 8 / MariaDB recente; diferenças entre versões são assinaladas.
Convenções: [ ] indica elemento opcional e não é escrito no SQL; nomes como nome_tabela são substituídos; ";" termina a instrução. Exemplos independentes podem exigir reposição da base de dados.

1. INDEX — Índices

CREATE INDEX

Para que serve?

Criar uma estrutura auxiliar para facilitar pesquisas; não cria uma coluna nem altera os registos.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`CREATE INDEX idx_paciente_nome ON Paciente (Nome);`	`CREATE [UNIQUE] INDEX nome_indice ON nome_tabela (coluna1 [, coluna2]);`

Resultado: Pode melhorar pesquisas por Nome, sobretudo com muitos registos.

Índice composto

Para que serve? **Apoiar pesquisas que usam mais do que uma coluna**.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`CREATE INDEX idx_consulta_data_hora ON Consulta (Data, Horario);`	`CREATE INDEX nome_indice ON nome_tabela (coluna1, coluna2);`

Resultado: Pode beneficiar pesquisas por Data, ou por Data e Horario; a ordem das colunas é relevante.

CREATE UNIQUE INDEX

Para que serve? **Impedir valores duplicados nos campos indexados**.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`CREATE UNIQUE INDEX idx_seguro_nome ON Seguros (Nome);`	`CREATE UNIQUE INDEX nome_indice ON nome_tabela (coluna);`

Resultado: Impede nomes de seguros repetidos, desde que não existam duplicados antes da criação.

Nota: UNIQUE não significa obrigatoriamente NOT NULL.

SHOW INDEX / DROP INDEX

Para que serve? **Inspecionar ou eliminar um índice secundário.**

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`SHOW INDEX FROM Paciente;` `DROP INDEX idx_paciente_nome ON Paciente;`	`SHOW INDEX FROM nome_tabela;` `DROP INDEX nome_indice ON nome_tabela;`

Resultado: O índice deixa de existir, mas os dados permanecem.

Nota: Para eliminar um índice normal, utiliza-se DROP INDEX. Para retirar uma chave primária, utiliza-se ALTER TABLE ... DROP PRIMARY KEY. Esta última operação pode não ser permitida se existirem chaves estrangeiras ou outras dependências.

Eliminar um índice não é o mesmo que eliminar uma chave primária. A chave primária é uma restrição de integridade que identifica unicamente cada registo da tabela.

Resumindo

O que queremos eliminar? Qual o comando?

O que queremos eliminar?	Comando
Um índice normal	`DROP INDEX nome_indice ON nome_tabela;`
Uma chave primária	`ALTER TABLE nome_tabela DROP PRIMARY KEY;`

Explicação simples

O **DROP INDEX** elimina uma estrutura que ajuda a pesquisar dados mais rapidamente. O **DROP PRIMARY KEY** retira a função de chave primária de um campo. Em ambos os casos, o campo continua a existir.

Nota

A expressão «sujeito a dependências» significa apenas que a chave primária pode não ser eliminada se outras tabelas precisarem dela para estabelecer relações.

TABELA RESUMO

1. INDEX — Índices

Os índices são estruturas auxiliares que permitem localizar os dados mais rapidamente, melhorando o desempenho das pesquisas.
Não criam colunas nem alteram os registos.

Operação	Para que serve?	Exemplo — Clínica Médica	Sintaxe geral	Resultado / Notas
CREATE INDEX	Criar uma estrutura auxiliar para facilitar pesquisas; não cria uma coluna nem altera os registos.	`CREATE INDEX idx_paciente_nome ON Paciente (Nome);`	`CREATE [UNIQUE] INDEX nome_indice ON nome_tabela (coluna1 [, coluna2]);`	Pode melhorar pesquisas por *Nome*, sobretudo com muitos registos.
Índice composto	Apoiar pesquisas que usam mais do que uma coluna.	`CREATE INDEX idx_consulta_data_hora ON Consulta (Data, Horario);`	`CREATE INDEX nome_indice ON nome_tabela (coluna1, coluna2);`	Pode beneficiar pesquisas por Data, ou por Data e Horario. A ordem das colunas é **relevante**.
CREATE UNIQUE INDEX	Impedir valores duplicados nos campos indexados.	`CREATE UNIQUE INDEX idx_seguro_nome ON Seguros (Nome);`	`CREATE UNIQUE INDEX nome_indice ON nome_tabela (coluna);`	Impede nomes de seguros repetidos, desde que não existam duplicados antes da criação. **Nota:** *UNIQUE* não significa obrigatoriamente *NOT NULL*.
SHOW INDEX / DROP INDEX	Inspecionar ou eliminar um índice secundário.	`SHOW INDEX FROM Paciente;` `DROP INDEX idx_paciente_nome ON Paciente;`	`SHOW INDEX FROM nome_tabela;` `DROP INDEX nome_indice ON nome_tabela;`	O índice deixa de existir, mas os dados permanecem. **Nota:** Para eliminar um índice normal, utiliza-se *DROP INDEX*. Para retirar uma chave primária, utiliza-se `ALTER TABLE ... DROP PRIMARY KEY`.

Exemplo prático — Pesquisa com e sem índice

Pesquisa de pacientes por nome:

Sem índice (pesquisa mais lenta) — Maior tempo de pesquisa

Cod_Paciente	Nome
1	Ana Silva
2	Bruno Costa
3	Carla Mendes
...	...

`CREATE INDEX idx_paciente_nome ON Paciente (Nome);`

Com índice (pesquisa mais rápida) — Menor tempo de pesquisa

Cod_Paciente	Nome
1	Ana Silva
2	Bruno Costa
3	Carla Mendes
...	...

Tipos de índices

Tipo	Descrição
Normal (INDEX)	Acelera pesquisas. Pode ter valores repetidos.
Composto	Usa mais do que uma coluna. A ordem das colunas é relevante.
Único (UNIQUE)	Acelera pesquisas e impede valores duplicados.
Primário (PRIMARY KEY)	Índice especial que identifica unicamente cada registo. Só pode existir um por tabela.

Notas importantes

- Eliminar um índice não é o mesmo que eliminar uma chave primária.
- A chave primária é uma restrição de integridade que identifica unicamente cada registo da tabela.
- **DROP INDEX** só elimina índices secundários (incluindo os criados com *UNIQUE*).
- Para eliminar uma chave primária utiliza-se: `ALTER TABLE nome_tabela DROP PRIMARY KEY;`
- A criação e eliminação de índices não altera os dados existentes.
- Índices melhoram o desempenho das pesquisas, mas podem aumentar o espaço em disco e o tempo de inserção/atualização.

2. ALTER TABLE — Alteração de tabelas

Para que serve?
Modificar a estrutura de uma tabela existente:

a) acrescentando,
b) alterando ou
c) eliminando colunas,
d) índices e
e) restrições, ou
f) renomeando a tabela.

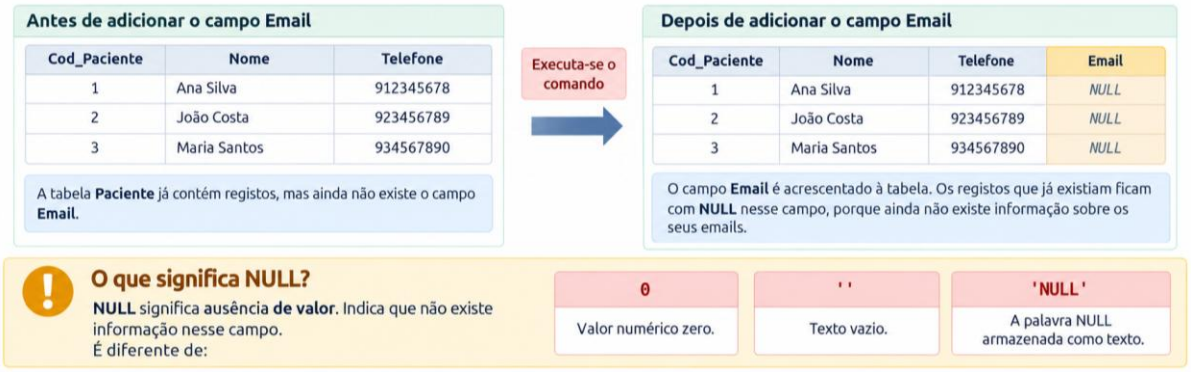
ALTER TABLE não é usado para alterar os valores dos registos (essa operação é UPDATE).

ADD [COLUMN]

Para que serve? Adicionar um campo.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`ALTER TABLE Paciente ADD COLUMN Email VARCHAR(100);`	`ALTER TABLE nome_tabela ADD [COLUMN] nome_coluna tipo [restricoes];`

Resultado: É adicionada a coluna Email à tabela Paciente. Os pacientes já registados ficam com NULL nesse campo, porque ainda não existe informação sobre os seus emails.



ADD várias colunas

Para que serve? Acrescentar mais do que um campo na mesma instrução.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`ALTER TABLE Paciente ADD Data_Nascimento DATE, ADD Observacoes VARCHAR(200);`	`ALTER TABLE nome_tabela ADD [COLUMN] coluna1 tipo, ADD [COLUMN] coluna2 tipo;`

Resultado: Acrescenta duas colunas.

MODIFY [COLUMN]

Para que serve? Alterar o tipo, tamanho ou outras propriedades de uma coluna, sem alterar o nome.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`ALTER TABLE Paciente MODIFY COLUMN Telefone VARCHAR(20);`	`ALTER TABLE nome_tabela MODIFY [COLUMN] nome_coluna nova_definicao;`

Resultado: Aumenta a capacidade do campo Telefone.

Nota: Ao redefinir uma coluna, voltar a declarar restrições que devam ser preservadas, como NOT NULL ou DEFAULT.

CHANGE [COLUMN]

Para que serve? Alterar o nome e/ou a definição de uma coluna; exige nome antigo, nome novo e definição.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`ALTER TABLE Paciente CHANGE COLUMN Endereco Morada VARCHAR(150);`	`ALTER TABLE nome_tabela CHANGE [COLUMN] nome_antigo nome_novo nova_definicao;`

Resultado: Renomeia Endereco para Morada e define VARCHAR(150).

RENAME COLUMN

Para que serve? Mudar apenas o nome da coluna. Aqui COLUMN é obrigatório.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`ALTER TABLE Paciente RENAME COLUMN Endereco TO Morada;`	`ALTER TABLE nome_tabela RENAME COLUMN nome_antigo TO nome_novo;`

Resultado: Renomeia o campo sem alterar explicitamente o tipo.

Nota: Disponibilidade depende da versão do servidor; **CHANGE** é alternativa em versões antigas.

DROP [COLUMN]

Para que serve? Eliminar uma coluna e os seus valores.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`ALTER TABLE Paciente DROP COLUMN Email;`	`ALTER TABLE nome_tabela DROP [COLUMN] nome_coluna;`

Resultado: Elimina a coluna Email; exige que exista e não esteja bloqueada por dependências.

SET DEFAULT

Para que serve? Definir o valor predefinido de uma coluna.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`ALTER TABLE Consulta ADD COLUMN Tipo_Consulta VARCHAR(30);` `ALTER TABLE Consulta ALTER COLUMN Tipo_Consulta SET DEFAULT 'Geral';`	`ALTER TABLE nome_tabela ALTER [COLUMN] nome_coluna SET DEFAULT valor;`

Resultado: Novas inserções que omitam Tipo_Consulta recebem por defeito a string: Geral.

DROP DEFAULT

Para que serve? Retirar um valor predefinido sem eliminar a coluna.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`ALTER TABLE Consulta ALTER COLUMN Tipo_Consulta DROP DEFAULT;`	`ALTER TABLE nome_tabela ALTER [COLUMN] nome_coluna DROP DEFAULT;`

Resultado: A coluna continua a existir.

ADD PRIMARY KEY

Para que serve? Definir chave primária numa tabela que ainda não a tenha.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`CREATE TABLE Sala (Cod_Sala INT NOT NULL, Designacao VARCHAR(50));` `ALTER TABLE Sala ADD PRIMARY KEY (Cod_Sala);`	`ALTER TABLE nome_tabela ADD PRIMARY KEY (coluna [, coluna2]);`

Resultado: Cod_Sala passa a identificar cada sala.

Nota: Os valores devem ser únicos e não nulos; só pode existir uma chave primária por tabela.

DROP PRIMARY KEY

Para que serve? Retirar a chave primária quando as dependências o permitem.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`ALTER TABLE Sala DROP PRIMARY KEY;`	`ALTER TABLE nome_tabela DROP PRIMARY KEY;`

Resultado: Remove a restrição de chave primária de Sala.

Nota: Executar apenas numa cópia de treino; **pode falhar por dependências ou AUTO_INCREMENT.**

ADD FOREIGN KEY

Para que serve? Criar uma relação de integridade referencial.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`ALTER TABLE Consulta ADD COLUMN Cod_Sala INT;` `ALTER TABLE Consulta ADD CONSTRAINT fk_consulta_sala FOREIGN KEY (Cod_Sala) REFERENCES Sala(Cod_Sala);`	`ALTER TABLE tabela ADD CONSTRAINT nome_fk FOREIGN KEY (campo) REFERENCES outra_tabela(campo);`

Resultado: Uma consulta passa a poder referenciar uma sala existente.

Nota: Executar após criar Sala com Cod_Sala como chave primária; os dados existentes têm de respeitar a relação.

DROP FOREIGN KEY

Para que serve? Eliminar uma restrição de chave estrangeira, mantendo as colunas.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`ALTER TABLE Consulta DROP FOREIGN KEY fk_consulta_sala;`	`ALTER TABLE nome_tabela DROP FOREIGN KEY nome_fk;`

Resultado: Retira a verificação da ligação à tabela Sala.

Nota: O índice de apoio poderá continuar a existir.

ADD UNIQUE / DROP INDEX

Para que serve? Criar ou remover uma restrição de unicidade (implementada por índice no MySQL).

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`ALTER TABLE Paciente ADD CONSTRAINT uq_paciente_email UNIQUE (Email);` `ALTER TABLE Paciente DROP INDEX uq_paciente_email;`	`ALTER TABLE tabela ADD CONSTRAINT nome UNIQUE (campo);` `ALTER TABLE tabela DROP INDEX nome;`

Resultado: Impede emails duplicados e, no segundo comando, retira essa restrição.

Nota: O campo Email deve existir; UNIQUE admite múltiplos NULL em MySQL.

ADD INDEX / DROP INDEX

Para que serve? Criar ou eliminar um índice pela instrução ALTER TABLE.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`ALTER TABLE Consulta ADD INDEX idx_data (Data);` `ALTER TABLE Consulta DROP INDEX idx_data;`	`ALTER TABLE tabela ADD INDEX nome (campo);` `ALTER TABLE tabela DROP INDEX nome;`

Resultado: Adiciona e remove o índice de pesquisas por data.

ADD CHECK / DROP CHECK

Para que serve? Validar uma condição para os dados. Definir ou eliminar uma regra que controla os valores permitidos num campo de uma tabela.

- ADD CHECK: cria uma regra que os dados devem respeitar.
- DROP CHECK: elimina essa regra, permitindo que deixem de existir as restrições anteriormente definidas.

Exemplo — Clínica Médica

Numa clínica, cada sala tem uma capacidade máxima de pessoas. Essa capacidade deve ser superior a zero, pois não faz sentido registar uma sala com capacidade negativa ou igual a zero.

1. Adicionar o campo Capacidade à tabela Sala

```
ALTER TABLE Sala
ADD COLUMN Capacidade INT;
```

2. Criar a regra de validação:

```
ALTER TABLE Sala
ADD CONSTRAINT chk_capacidade
CHECK (Capacidade > 0);
```

3. Testar a regra:

```
INSERT INTO Sala (Cod_Sala, Capacidade)
VALUES (1, -5);
```

O que acontece na prática?

Valor introduzido	Resultado	Explicação
20	✓ Aceite	Superior a zero
5	✓ Aceite	Superior a zero
0	✗ Rejeitado	Não é superior a zero
-3	✗ Rejeitado	Valor negativo
NULL	! Permitido	CHECK não impede valores NULL

Nota: o teste pressupõe que a tabela Sala já existe, que o código 1 ainda não está atribuído e que não existem outros campos obrigatórios por preencher.

Resultado esperado: O MariaDB rejeita a inserção porque -5 não respeita a condição **Capacidade > 0**.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`ALTER TABLE Sala ADD COLUMN Capacidade INT;` `ALTER TABLE Sala ADD CONSTRAINT chk_capacidade CHECK (Capacidade > 0);`	`ALTER TABLE tabela ADD CONSTRAINT nome CHECK (condicao);` `ALTER TABLE tabela DROP CHECK nome;`

Resultado: Impede capacidades não positivas em versões que aplicam CHECK.
Nota: MySQL anterior a 8.0.16 aceitava CHECK mas não o aplicava; verificar versão.

Atenção à diferença entre MySQL e MariaDB

Sistema	Eliminar CHECK
MySQL 8.0.16+	DROP CHECK chk_capacidade
MariaDB (XAMPP)	DROP CONSTRAINT chk_capacidade

Para verificar a versão utilizada no teu XAMPP, executa: SELECT VERSION();

RENAME TO

Para que serve? mudar o nome de uma tabela que já existe na base de dados, sem eliminar os seus campos ou registos.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`ALTER TABLE Seguros RENAME TO Seguro;`	`ALTER TABLE nome_antigo RENAME TO nome_novo;`

Resultado: A tabela passa a chamar-se Seguro.

Nota: Resultado: A tabela Seguros passa a chamar-se Seguro. Os campos e os registos mantêm-se. A partir deste momento, os comandos SQL devem utilizar o novo nome Seguro.

TABELA RESUMO

ALTER TABLE — RESUMO DAS PRINCIPAIS OPERAÇÕES

Para que serve?	Modificar a estrutura de uma tabela existente: acrescentando, alterando ou eliminando colunas, índices e restrições, ou renomeando a tabela. ALTER TABLE não é usado para alterar os valores dos registos (essa operação é UPDATE).

Operação	Para que serve?	Exemplo — Clínica Médica	Sintaxe geral	Resultado / Notas
ADD [COLUMN]	Adicionar um campo.	`ALTER TABLE Paciente ADD COLUMN Email VARCHAR(100);`	`ALTER TABLE nome_tabela ADD [COLUMN] nome_coluna tipo [restricoes];`	Cria a coluna **Email**. Registos anteriores ficam com *NULL* se o campo admitir *NULL*.
ADD várias colunas	Acrescentar mais do que um campo na mesma instrução.	`ALTER TABLE Paciente ADD Data_Nascimento DATE, ADD Observacoes VARCHAR(200);`	`ALTER TABLE nome_tabela ADD [COLUMN] coluna1 tipo, ADD [COLUMN] coluna2 tipo;`	Acrescenta duas colunas.
MODIFY [COLUMN]	Alterar o tipo, tamanho ou outras propriedades de uma coluna, sem alterar o nome.	`ALTER TABLE Paciente MODIFY COLUMN Telefone VARCHAR(20);`	`ALTER TABLE nome_tabela MODIFY [COLUMN] nome_coluna nova_definicao;`	Altera a definição da coluna. **Nota:** voltar a declarar restrições como *NOT NULL* ou *DEFAULT*.
CHANGE [COLUMN]	Alterar o nome e/ou a definição de uma coluna. Exige nome antigo, nome novo e definição.	`ALTER TABLE Paciente CHANGE COLUMN Endereco Morada VARCHAR(150);`	`ALTER TABLE nome_tabela CHANGE [COLUMN] nome_antigo nome_novo nova_definicao;`	Renomeia e define a nova coluna.
RENAME COLUMN	Mudar apenas o nome da coluna (*COLUMN* é obrigatório).	`ALTER TABLE Paciente RENAME COLUMN Endereco TO Morada;`	`ALTER TABLE nome_tabela RENAME COLUMN nome_antigo TO nome_novo;`	Renomeia a coluna, mantendo o tipo. **Nota:** depende da versão do servidor; *CHANGE* é alternativa em versões antigas.
DROP [COLUMN]	Eliminar uma coluna e os seus valores.	`ALTER TABLE Paciente DROP COLUMN Email;`	`ALTER TABLE nome_tabela DROP [COLUMN] nome_coluna;`	Elimina a coluna. **Nota:** a coluna deve existir e não pode estar bloqueada por dependências.
SET DEFAULT	Definir o valor predefinido de uma coluna.	`ALTER TABLE Consulta ALTER COLUMN Tipo_Consulta SET DEFAULT 'Geral';`	`ALTER TABLE nome_tabela ALTER [COLUMN] nome_coluna SET DEFAULT valor;`	Novas inserções recebem o valor predefinido.
DROP DEFAULT	Retirar um valor predefinido da coluna.	`ALTER TABLE Consulta ALTER COLUMN Tipo_Consulta DROP DEFAULT;`	`ALTER TABLE nome_tabela ALTER [COLUMN] nome_coluna DROP DEFAULT;`	Remove o valor predefinido. A coluna continua a existir.
ADD PRIMARY KEY	Definir chave primária numa tabela.	`ALTER TABLE Sala ADD PRIMARY KEY (Cod_Sala);`	`ALTER TABLE nome_tabela ADD PRIMARY KEY (coluna [, coluna2]);`	Define a chave primária. Valores devem ser únicos e não nulos. Só pode existir uma chave primária.
DROP PRIMARY KEY	Retirar a chave primária.	`ALTER TABLE Sala DROP PRIMARY KEY;`	`ALTER TABLE nome_tabela DROP PRIMARY KEY;`	Remove a chave primária. **Nota:** pode falhar por dependências ou *AUTO_INCREMENT*.
ADD FOREIGN KEY	Criar uma relação de integridade referencial.	`ALTER TABLE Consulta ADD CONSTRAINT fk_consulta_sala FOREIGN KEY (Cod_Sala) REFERENCES Sala(Cod_Sala);`	`ALTER TABLE tabela ADD CONSTRAINT nome_fk FOREIGN KEY (campo) REFERENCES outra_tabela(campo);`	Cria uma chave estrangeira. Os dados existentes têm de respeitar a relação.
DROP FOREIGN KEY	Eliminar uma restrição de chave estrangeira.	`ALTER TABLE Consulta DROP FOREIGN KEY fk_consulta_sala;`	`ALTER TABLE nome_tabela DROP FOREIGN KEY nome_fk;`	Remove a restrição. O índice de apoio poderá continuar.
ADD UNIQUE DROP INDEX	Criar ou remover uma restrição de unicidade.	`ALTER TABLE Paciente ADD CONSTRAINT uq_paciente_email UNIQUE (Email); ALTER TABLE Paciente DROP INDEX uq_paciente_email;`	`ALTER TABLE tabela ADD CONSTRAINT nome UNIQUE (campo); ALTER TABLE tabela DROP INDEX nome;`	Impede valores duplicados. **Nota:** em MySQL, *UNIQUE* admite múltiplos *NULL*.
ADD INDEX DROP INDEX	Criar ou eliminar um índice.	`ALTER TABLE Consulta ADD INDEX idx_data (Data); ALTER TABLE Consulta DROP INDEX idx_data;`	`ALTER TABLE tabela ADD INDEX nome (campo); ALTER TABLE tabela DROP INDEX nome;`	Adiciona/remove um índice para pesquisas.
ADD CHECK DROP CHECK	Validar uma condição para os dados.	`ALTER TABLE Sala ADD CONSTRAINT chk_capacidade CHECK (Capacidade > 0);`	`ALTER TABLE tabela ADD CONSTRAINT nome CHECK (condicao); ALTER TABLE tabela DROP CHECK nome;`	Impede valores inválidos. **Nota:** em MySQL anterior a 8.0.16, o *CHECK* era aceite mas não aplicado.
RENAME TO	Mudar o nome da tabela.	`ALTER TABLE Seguros RENAME TO Seguro;`	`ALTER TABLE nome_antigo RENAME TO nome_novo;`	A tabela passa a chamar-se **Seguro**. Os campos e os registos mantêm-se. Atualizar referências nos comandos SQL.

COMPARAÇÃO ESSENCIAL — MODIFY, CHANGE e RENAME COLUMN

Operação	Altera o nome da coluna?	Altera o tipo/tamanho/restrições?	Descrição	Exemplo — Clínica Médica
MODIFY [COLUMN]	Não	Sim	Altera o tipo, tamanho ou outras propriedades da coluna (mantém o nome).	`ALTER TABLE Paciente MODIFY COLUMN Telefone VARCHAR(20);` (mantém o nome *Telefone*)
CHANGE [COLUMN]	Pode	Sim	Altera o nome e/ou a definição da coluna.	`ALTER TABLE Paciente CHANGE COLUMN Telefone Contacto VARCHAR(20);` (muda o nome e o tipo)
RENAME COLUMN	Sim	Não	Altera apenas o nome da coluna (mantém o tipo).	`ALTER TABLE Paciente RENAME COLUMN Telefone TO Contacto;` (só muda o nome)

3. Gestão de utilizadores e privilégios

Aplicabilidade: numa clínica, as contas da receção, administração e manutenção devem ter acessos diferentes. O princípio do menor privilégio significa conceder apenas o necessário para cada função. Estes comandos exigem permissões administrativas e geralmente não funcionam em editores SQL públicos.

CREATE USER

Para que serve? Criar uma conta MySQL, não um registo na tabela Paciente.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`CREATE USER 'rececionista'@'localhost' IDENTIFIED BY 'ExemploTemporario#123';`	`CREATE USER 'utilizador'@'host' IDENTIFIED BY 'palavra_passe';`

Resultado: Cria uma conta para ligações locais; a palavra-passe é fictícia.

GRANT

Para que serve? Atribuir permissões específicas numa tabela.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`GRANT SELECT, INSERT ON ClinicaMedica.Paciente TO 'rececionista'@'localhost';`	`GRANT privilegio [, privilegio] ON base.tabela TO 'utilizador'@'host';`

Resultado: Permite consultar e inserir pacientes, mas não atualizar ou eliminar.

SHOW GRANTS

Para que serve? Consultar privilégios efetivos atribuídos à conta.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`SHOW GRANTS FOR 'rececionista'@'localhost';`	`SHOW GRANTS FOR 'utilizador'@'host';`

Resultado: Mostra as permissões concedidas.

REVOKE

Para que serve? Retirar um privilégio concedido.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`REVOKE INSERT ON ClinicaMedica.Paciente FROM 'rececionista'@'localhost';`	`REVOKE privilegio ON base.tabela FROM 'utilizador'@'host';`

Resultado: Retira a permissão de inserir pacientes; SELECT mantém-se.

ALTER USER

Para que serve? Modificar propriedades da conta, por exemplo a palavra-passe.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`ALTER USER 'rececionista'@'localhost' IDENTIFIED BY 'OutroExemplo#456';`	`ALTER USER 'utilizador'@'host' IDENTIFIED BY 'nova_palavra_passe';`

Resultado: Atualiza a palavra-passe da conta.

DROP USER

Para que serve? Eliminar a conta, sem eliminar dados da clínica.

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL
`DROP USER 'rececionista'@'localhost';`	`DROP USER 'utilizador'@'host';`

Resultado: Remove a conta e os respetivos privilégios.

TABELA RESUMO



Notas importantes: CREATE USER cria uma conta de acesso ao SGBD, não um paciente. DROP USER elimina a conta, mas não os dados das tabelas. Estes comandos exigem permissões adequadas e geralmente não podem ser testados em editores SQL públicos.

Atenção: nos exemplos foi utilizado ClinicaMedica como nome da base de dados. Se estiveres a utilizar a base de dados Clinica, criada nos exercícios anteriores, deves substituir ClinicaMedica.Paciente por Clinica.Paciente.

O princípio do menor privilégio consiste em conceder a cada utilizador apenas os acessos necessários para realizar o seu trabalho.

4. COMMIT | ROLLBACK — Transações (Savepoint)

Tutorial prático de SQL | Módulo 15 | Clínica Médica

Objetivo: compreender como confirmar ou anular alterações aos dados utilizando transações em MySQL/MariaDB.

1. O que é uma transação?

Uma transação é um conjunto de operações SQL tratado como uma unidade de trabalho. Podemos confirmar as alterações com COMMIT ou anulá-las com ROLLBACK, desde que ainda não tenham sido confirmadas.

Comando	Função	Ideia-chave
START TRANSACTION	Inicia uma transação.	Começar
COMMIT	Confirma as alterações da transação.	Guardar
ROLLBACK	Anula as alterações ainda não confirmadas.	Voltar atrás

Comando	Função
START TRANSACTION	Inicia uma transação.
SELECT ... FOR UPDATE	Bloqueia o registo selecionado para atualização.
UPDATE	Altera a quantidade em stock.
COMMIT	Confirma as alterações e liberta o bloqueio.
ROLLBACK	Anula alterações não confirmadas e termina a transação.

Conteúdo do M14

2. Sintaxe e exemplo da aplicabilidade dos comandos – Exemplo a analisar:

EXEMPLO — CLÍNICA MÉDICA	SINTAXE GERAL

`START TRANSACTION;`
`  UPDATE Paciente`
`  SET Telefone = '912345678'`
`  WHERE Cod_Paciente = 1;`
`  COMMIT;`
	`START TRANSACTION;`
	`  comando(s) SQL;`
	`  COMMIT;`

`START TRANSACTION;`
`  UPDATE Paciente`
`  SET Telefone = '912345678'`
`  WHERE Cod_Paciente = 1;`
`  ROLLBACK;`
	`START TRANSACTION;`
	`  comando(s) SQL;`
	`  ROLLBACK;`

3. Antes e depois: alterar um telefone

Suponhamos que o paciente 1 tem inicialmente o telefone 911111111.

Momento	Cod_Paciente	Nome	Telefone
Antes da transação	1	Ana Silva	911111111
Depois de COMMIT	1	Ana Silva	912345678
Depois de ROLLBACK (alternativa)	1	Ana Silva	911111111

Atenção: COMMIT e ROLLBACK representam dois finais alternativos para a mesma situação. Não devem ser executados consecutivamente à espera de que ROLLBACK desfaça um COMMIT.

4. Experiência A — confirmar a alteração

Exemplo — Clínica Médica

```
1  START TRANSACTION;
2  UPDATE Paciente
3  SET Telefone = '912345678'
4  WHERE Cod_Paciente = 1;
5  COMMIT;

7  SELECT Cod_Paciente, Nome, Telefone
8  FROM Paciente WHERE Cod_Paciente = 1;
```

Executa-se o comando

Experiência A — confirmar a alteração

Resultado esperado
O novo telefone fica confirmado na base de dados.

Cod_Paciente	Nome	Telefone
1	Ana Silva	912345678

Nota: o resultado mostra o novo telefone, confirmando que a alteração foi guardada na base de dados.

5. Experiência B — anular a alteração

Partindo novamente do valor inicial, ou usando outro telefone para testar:

Exemplo — Clínica Médica | Experiência B — anular a alteração (ROLLBACK)

```
1  START TRANSACTION;
2  UPDATE Paciente
3  SET Telefone = '9999999999'
4  WHERE Cod_Paciente = 1;
5  ROLLBACK;

7  SELECT Cod_Paciente, Nome, Telefone
8  FROM Paciente WHERE Cod_Paciente = 1;
```

Executa-se o comando

Resultado esperado
O valor 999999999 não fica guardado; mantém-se o telefone que existia antes desta transação.

Cod_Paciente	Nome	Telefone
1	Ana Silva	911111111

Nota: o *ROLLBACK* anula todas as alterações feitas nesta transação, por isso o telefone mantém-se com o valor anterior (911111111).

6. Exemplo real: registo de uma consulta

Uma clínica pretende registar uma consulta. Se o registo estiver correto, confirma-o; se tiver sido criado por engano, anula-o.

Exemplo — Clínica Médica | Registo de uma consulta (COMMIT ou ROLLBACK)

Uma clínica pretende registar uma consulta. Se o registo estiver correto, confirma-o; se tiver sido criado por engano, anula-o.

```
1  START TRANSACTION;
2  INSERT INTO Consulta (Data, Horario, Cod_Paciente, Cod_Med)
3  VALUES ('2026-10-20', '10:30:00', 1, 1);
4
5  -- Escolher apenas uma das opções:
6  COMMIT;
7  -- ou ROLLBACK; (em vez de COMMIT)
8
```

Se executar COMMIT
A consulta fica registada na tabela Consulta.

Data	Horario	Cod_Paciente	Cod_Med
2026-10-20	10:30:00	1	1

Se executar ROLLBACK
A consulta não fica guardada na tabela Consulta.

Data	Horario	Cod_Paciente	Cod_Med
Sem novos registos.			

Nota: Os códigos 1 de paciente e médico devem existir previamente, devido às chaves estrangeiras.

7. Cuidados importantes

- Os exemplos pressupõem tabelas InnoDB, que suportam transações.
- Em MySQL/MariaDB, com autocommit ativo, cada alteração isolada pode ser confirmada automaticamente; START TRANSACTION permite controlar o conjunto de operações.
- Depois de COMMIT, um ROLLBACK posterior não desfaz essa transação.
- CREATE TABLE, ALTER TABLE e outros comandos DDL podem provocar confirmação implícita; não são adequados para demonstrar ROLLBACK.

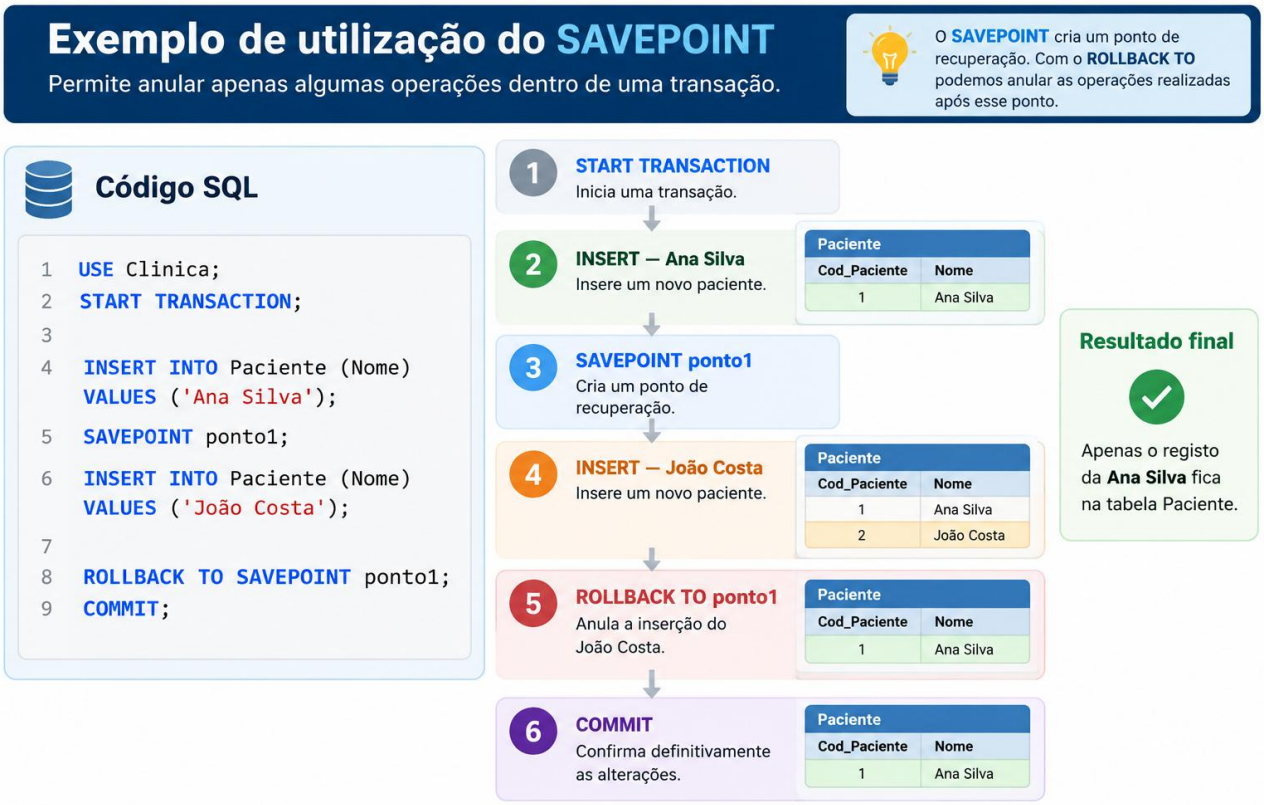
SAVEPOINT - Criar pontos de recuperação numa transação

O que é?

O SAVEPOINT cria um ponto de recuperação dentro de uma transação. Permite anular apenas as operações realizadas depois desse ponto, sem anular todas as operações anteriores.

Exemplo na base de dados Clinica

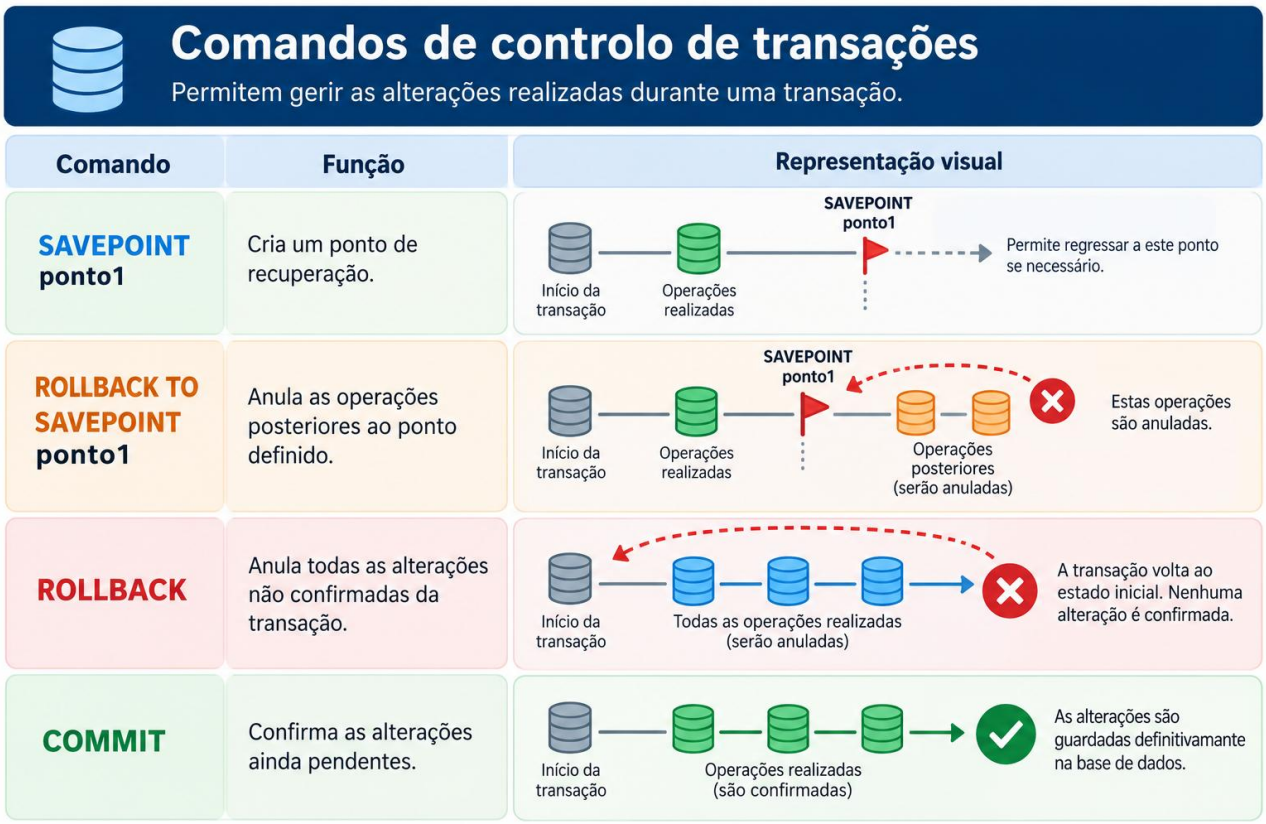
Pretende-se registar dois pacientes, mas cancelar o registo do segundo antes de confirmar a transação.



Resultado esperado

- Ana Silva: o registo é confirmado pelo COMMIT.
- João Costa: o registo é anulado pelo ROLLBACK TO SAVEPOINT.

Diferença entre os commandos



Importante

O SAVEPOINT não grava definitivamente os dados.

Só o COMMIT confirma a transação. Depois de um COMMIT ou de um ROLLBACK completo, os pontos de recuperação deixam de estar disponíveis. Este exemplo pressupõe tabelas transacionais, como InnoDB.